



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Performance Prediction and Preselection for Optimization and Heuristic Solution Procedures

Nicholas G. Hall, Marc E. Posner,

To cite this article:

Nicholas G. Hall, Marc E. Posner, (2007) Performance Prediction and Preselection for Optimization and Heuristic Solution Procedures. Operations Research 55(4):703-716. <https://doi.org/10.1287/opre.1070.0398>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2007, INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Performance Prediction and Preselection for Optimization and Heuristic Solution Procedures

Nicholas G. Hall

Fisher College of Business, The Ohio State University, Columbus, Ohio 43210, hall.33@osu.edu

Marc E. Posner

Integrated Systems Engineering, The Ohio State University, Columbus, Ohio 43210, posner.1@osu.edu

The operations research literature contains numerous studies on the design and application of optimization and heuristic solution procedures. These studies identify a particular optimization problem, suggest a general solution procedure, and then customize that procedure to improve its efficiency and/or accuracy. In contrast, this paper shows how to use existing solution procedures more effectively. We develop a methodology for predicting the relative performance of alternative procedures, using easily computed problem characteristics. This methodology enables us, for any given data set, to preselect a solution procedure. We apply this preselection methodology to the 0-1 knapsack problem for which two successful optimization procedures, dynamic programming and branch-and-search, are available. Extensive computational testing indicates that substantial savings in average computation time are achieved. The benefits of our work include faster and cheaper identification of effective solution procedures, as well as an improved understanding of the relationship between problem characteristics and the performance of various procedures. Our methodology can be applied to many optimization problems to develop easily implemented guidelines for selecting appropriate solution procedures.

Subject classifications: computational testing; prediction of solution procedure performance; choice of solution procedure; statistical analysis; programming: integer, algorithms, heuristics; analysis of algorithms: suboptimal algorithms; statistics: data analysis.

Area of review: Optimization

History: Received October 2004; revision received September 2005; accepted March 2006.

1. Introduction

A large part of the substantial literature of operations research focuses on *optimization*. Optimization is broadly defined to include the design and evaluation of solution procedures that identify either an optimal or an approximate solution. The operations research literature provides numerous applications of optimization in manufacturing, logistics, finance, marketing, public policy, military, health care, and other problems which require complex decision making.

Many optimization studies begin by identifying an interesting application. Often the literature suggests several competitive solution procedures. There are few guidelines about which solution procedure is best suited for a particular problem instance. Even when such guidelines exist, if the types of data sets that arise in a newly identified application are substantially different from those found in published computational studies, then the earlier results may be irrelevant or even misleading for the new application (Hall and Posner 2001). This is due to the fact that most optimization procedures work well for some types of data sets and poorly for others.

Our research provides a remedy for this problem. Instead of trying to make small improvements to an existing solution procedure, we focus on *making better choices between*

existing procedures. We provide a general methodology for quickly determining which procedure to use for a specific problem instance. We show that using this methodology can result in substantial reductions in computation time.

There is a related literature that considers factors which influence problem solvability. Johnson (1983) discusses ways in which the introduction of constraints into problem generation techniques affects the computational complexity of a problem. Cheeseman et al. (1991) investigate the phenomenon that typical instances of many intractable problems are easy to solve. They discuss the existence of one or more parameters that have critical ranges, within which most difficult problem instances occur.

By studying the autocorrelation coefficient of a cost function, Angel and Zissimopoulos (1998) provide support for empirically observed local search results. The performance of a local search procedure can be predicted from the autocorrelation coefficient, the maximum distance between any two solutions, and the size of the neighborhood. Angel and Zissimopoulos (2000) describe a hierarchy of combinatorial optimization problems, based on their autocorrelation coefficients.

The problem of minimizing the weighted number of late jobs scheduled nonpreemptively on a single machine is relevant to our work because both dynamic programming and

branch-and-search solution procedures are available. For this problem, Potts and Van Wassenhove (1988) develop dominance rules and other problem reduction techniques. The reduced problem is solved using both dynamic programming and branch-and-search with the incorporation of improved lower bounds. The authors conclude that these two general procedures are competitive for many problems. However, for problems with particularly difficult structure, dynamic programming appears to work better.

Szwarc et al. (2001) study the single-machine total tardiness problem, for which both of the above optimization procedures are available. They identify two apparent computational paradoxes. First, deleting a bounding rule reduces the size of the enumeration tree. Second, using a stronger decomposition rule degrades performance.

The 0-1 knapsack problem (Papadimitriou and Steiglitz 1982) is also solvable using either dynamic programming or branch-and-search. Balas and Zemel (1980) describe and test an implicit enumeration algorithm. The *core* of the problem is an important concept in their analysis. The core is defined as the subset of items whose value-to-size ratio falls between the maximum and minimum ratios of items that have different optimal and fractional solutions. The core defines the approximate amount of computation required to solve the problem. However, this concept is difficult to operationalize because the optimal solution is unknown. For the same problem, Martello and Toth (1984) describe a hybrid of the two optimization procedures discussed above, using depth-first-search dynamic programming.

Martello and Toth (1988) incorporate new heuristic upper bounding techniques and problem reduction procedures into another algorithm for the 0-1 knapsack problem. Martello and Toth (1997) develop a Lagrangian relaxation scheme and valid inequalities. Pisinger (1997) evaluates a breadth-first-search dynamic program in which the enumeration of the core is performed iteratively by adding new variables as needed. Pisinger (1999) further investigates the core. Hill and Reilly (2002) study the effects of coefficient structure and constraint tightness on the performance of branch-and-search for the closely related two-dimensional knapsack problem. They show that the factor which most significantly affects performance is the correlation structure between the two constraints.

To choose between procedures, this work presents a general methodology that uses instance specific information. This methodology encourages the user to examine the various factors that influence a particular selection. Techniques are described for predicting the relative performance of various procedures, based on easily computed characteristics of the problem instance, without the need for extensive computational testing. However, we do not elaborate on the details of the techniques because they vary between problems and between solution procedures.

We illustrate the methodology by developing an approach to select a single optimization procedure, from

either dynamic programming or branch-and-search, for a given instance of the 0-1 knapsack problem. As part of our analysis, a heuristic is developed to estimate the computation time of the branch-and-search procedure. For a given instance, we predict with high probability which optimization procedure is likely to execute more quickly. Our computational results show that our predictions achieve a 60.7% reduction in mean computation time, relative to choosing either optimization procedure. The best possible reduction, given a hypothetical perfect preselection procedure, is 81.5%.

Because we are studying the behavior of solution procedures under a variety of conditions, one outcome of our analysis is an identification of the strengths and weaknesses of a given procedure. Using the actual data, we can better understand the strengths and limitations of the procedure's ability to solve the instance. Consequently, our work also provides an improved understanding of the relationship between problem characteristics and the relative performance of various procedures.

This paper is organized as follows. In §2, we discuss the foundation of a general methodology to select between competing procedures. Details of the methodology are presented in §3. Sections 4 through 6 show how this methodology can be applied to select a single optimization procedure for the 0-1 knapsack problem. Section 4 provides notation and a problem description. Also, generic dynamic programming and branch-and-search procedures are described. In §5, we identify and justify the characteristics that are expected to be useful in predicting solution procedure performance for the 0-1 knapsack problem. Based on the analysis of an extensive test set of problems, we describe a way to predict which of these procedures is likely to perform better for a particular problem instance. In §6, our predictions are tested against numerous randomly generated problem instances. The results indicate that, even in parameter ranges where the two optimization procedures are competitive, we are able to preselect a procedure which on average performs much better than using either procedure for all instances. Finally, §7 provides a conclusion and some suggestions for future research.

2. Attributes and Measures

We first discuss the *performance objective(s)* that are used to select a solution procedure. For optimization procedures, the usual objective is to find an optimal solution as quickly as possible. For heuristic procedures, there are often two performance objectives. Usually the more important one is to find a solution with a value that is close to optimal. The second objective is to find a solution quickly (see for example, Neto 1999). Except for rare cases such as the shifting bottleneck procedure of Adams et al. (1988), heuristics that have exponential time complexity are neither regarded favorably in the literature nor widely used in practice. Other performance objectives are also possible.

The key factors of the data that may influence the performance of a given procedure for a given problem are called *attributes*. For example, in the traveling salesman problem, one attribute of a data set is whether the cities are clustered. This attribute is potentially useful because there is strong empirical evidence to suggest that many heuristic procedures have greater difficulty finding good solutions when there is clustering (Johnson and McGeogh 1997, 2002a; Neto 1999).

The attributes of the data may vary depending on the generality of the problem studied. As an example, most heuristic procedures find the symmetric version of the traveling salesman problem easier to solve than the asymmetric version (Johnson and McGeogh 1997, 2002a, b). If solution procedures are being compared for the symmetric problem, then symmetry is not an attribute of the data because symmetry is a common characteristic of all data sets. However, if the asymmetric case is being studied, then the amount of symmetry is potentially an attribute of the data. While there is apparently no work that examines the effects of symmetry in detail, we conjecture that many solution procedures perform more effectively as the amount of symmetry increases.

To operationalize the notion of an attribute, we need to quantify it. An attribute may be quantified for one or more of the following reasons:

- (1) To control a causative variable, in order to determine the effect of the attribute on a particular solution procedure.
- (2) To ensure that the data varies so as not to bias an experiment.
- (3) To document data variation that occurs naturally in practice.

First, we may want to test whether the attribute affects the performance of a solution procedure. To do this, we need to identify changes in the attribute between data sets. For example, we may be interested in how the variation of the transportation costs affects the efficiency of a branch-and-bound procedure for the uncapacitated plant location problem, or how the density of the graph affects the accuracy of a heuristic for the maximum clique problem.

Second, the data may inadvertently be generated in a way that systematically biases the analysis. For example, to ensure feasibility, a large amount of slack may be introduced into most of the constraints. By evaluating the amount of slack in an instance, it is possible to ascertain whether we are generating a set of instances where the amount of slack is too large. Evaluation of an attribute may also be needed for reasons other than determining average case performance. It may affect an order of complexity, a worst case performance bound, or a speed of convergence. Consider robotic cell scheduling, for example. The relative speeds of the robot and the machines affect the computational complexity of many problems. If robot move times are longer than processing times, then scheduling problems are solvable in polynomial time; whereas, if robot move

times are relatively short, then scheduling problems are NP-hard (Hall et al. 1997). Another example is the set covering problem. The worst case error of a greedy heuristic is bounded above by a function of the maximum column sum of the constraint matrix (Chvatal 1979), or by the maximum row sum of the constraint matrix (Bar Yehuda and Even 1981). Finally, for the planar traveling salesman problem, Karp (1977) uses the size of a rectangle as an attribute to establish the speed of convergence of the value found by a heuristic to the optimal value. In this case, smaller rectangles provide faster convergence.

Third, if there are specific types of problems that occur in practice, then we need to generate instances that reflect these types of problems. By making statistical comparisons between the data attributes of real-world problems and our generated problems, we can determine whether our generation scheme captures the relevant problem conditions. For example, if a company needs to solve a vehicle routing problem, then it is most interested in performance for data sets where the properties of the model tested are similar to those in the transport region. While all three reasons discussed above are important, the focus of this research is on the first one.

The reasons discussed in the previous paragraph establish the need for a way to quantify an attribute. A *measure* is a way to accomplish this. A measure maps an attribute to a numerical value. For example, the mean and variance are measures that evaluate the attributes of average value and dispersion, respectively. These values can be computed using standard statistical methods. Another example of an attribute is the tightness of the constraints for the set covering problem. One measure of constraint tightness is the number of different feasible covers.

Sometimes, several measures are required to describe an attribute. Consider the previously discussed attribute of node clustering for the traveling salesman problem. To determine whether a data set is highly clustered, we need to measure the degree of clustering. However, to describe the attribute fully, it appears that it is necessary to measure the within cluster variance, the between cluster variance, and the number of clusters. It may be the case that, for some solution procedures, the hardest data sets are those where there are only two tightly packed clusters. Only a thorough testing of the procedures for appropriate data sets can accurately identify the relationships between the measures and the performance of the solution procedures.

Some characteristics that a measure should have are now listed and briefly discussed. These include:

- (1) relevance to the experiment,
- (2) appropriate change, and
- (3) tractability.

First, the measure should quantify the attributes of a data set in a way that is relevant to the procedures being tested and to the problem. For the 0-1 knapsack problem, the attribute *knapsack capacity* can be quantified in several ways. For dynamic programming, the actual size of

the knapsack is most meaningful because computation time directly depends on it. However, for branch-and-search, the approximate number of items that can be held in the knapsack is more meaningful. This is because the number of feasible solutions that branch-and-search enumerates is directly related to this measure.

Another example of using a relevant measure occurs with bottleneck optimization problems. As discussed above, we might use the variance of the intercity distances as a measure for the traveling salesman problem. However, in the bottleneck traveling salesman problem (Garfinkel and Gilbert 1978), the objective is to find a tour that minimizes the maximum distance between any pair of cities. Here, the number of distinct intercity distances is a more relevant measure than the variance because the former measure also defines the number of possible optimal solution values.

Second, the measure should not change spuriously with respect to the problem. For example, in the set covering problem, using the ratio of the average variable cost to the average number of constraints covered by a variable is not a good measure for most procedures. This is because if all costs change in proportion, then the relative costs of all solutions are unchanged. Consequently, the performance of many solution procedures is unaffected.

Third, because the purpose of a measure is to provide information for selecting a solution procedure for a problem instance, the evaluation of a measure should generally take significantly less time than it takes to perform the procedure itself. For example, calculating the actual number of feasible schedules is NP-hard for many optimization problems. However, it is possible to use a *surrogate measure*, such as the expected number of feasible solutions. This value can sometimes be calculated from the generation process. For example, in the graph coloring problem, the expected number of feasible solutions can be computed using the number of edges in the graph as a measure (Williams and Hogg 1994).

There are three ways in which a surrogate measure can estimate the actual measure:

- (a) estimate the expected value of the actual measure,
- (b) provide a lower bound on the actual measure, or
- (c) provide an upper bound on the actual measure.

An example of an expected value estimate is given above in the calculation of the expected number of feasible schedules. An example of an upper bound appears in our study of the 0-1 knapsack problem. We observe that the number of nodes evaluated by the branch-and-search procedure is a very accurate measure of computational performance. However, the exact calculation of this measure is as computationally intensive as the work performed by branch-and-search to find an optimal solution. Furthermore, because the computation times of the branch-and-search procedure are highly skewed, it is important to identify those data sets where the number of nodes evaluated may be large. Consequently, we develop a surrogate measure that quickly provides an approximate upper bound on the number of nodes

that the branch-and-search procedure evaluates. Details are provided in §5.2.

3. Methodology for Selecting Solution Procedures

While in this section we describe a general methodology for selecting a solution procedure, we provide only guidelines. Similar to many general procedures for integer programming, the details need to be customized for each specific situation. We propose the following method for preselecting a solution procedure.

Method Preselect

Step 1. Determine the performance objectives of the solution procedures.

Step 2. Based on the performance objectives and the details of the solution procedures, identify attributes of the data.

Step 3. For each procedure, identify appropriate measures to evaluate each relevant attribute.

Step 4. Calibrate the measures with the performance of each procedure.

Step 5. Construct an appropriate data set, and experimentally develop and fine tune a set of detailed decision rules for preselecting a procedure.

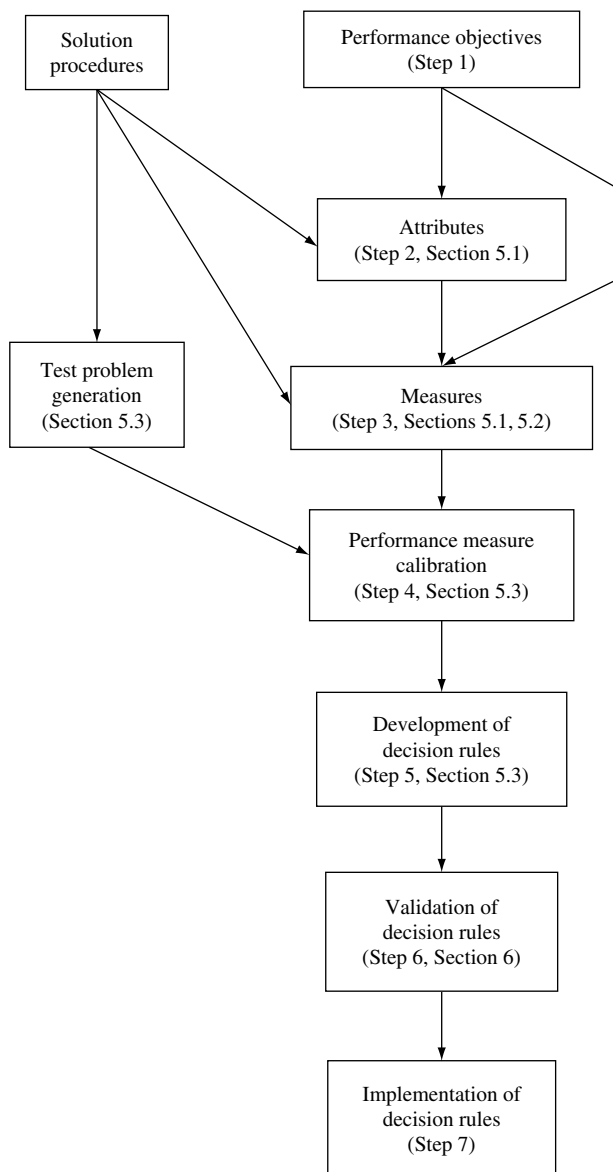
Step 6. Validate the decision rules and their contributions.

Step 7. For each specific data set, implement the decision rules to select a solution procedure, and run the selected procedure.

A flow diagram that shows the steps of Method Preselect appears in Figure 1. Where relevant, the elements in Figure 1 show the corresponding step number and a reference to where in the paper a detailed discussion can be found. As mentioned in §2, the most common objectives in Step 1 of Method Preselect are computation time for optimization procedures and accuracy of solution for heuristics. However, computation time may also be a consideration in the selection of a heuristic procedure. Other possible objectives include the storage space required to solve the problem, and the sensitivity of the solution to parameter changes. With respect to the objectives just mentioned, we may be interested in the average case performance, the worst case performance, or the variance of the performance. For example, solving a knapsack problem by dynamic programming may take longer on average than using branch-and-search, but it may be preferable because there are no instances that take an excessively long time.

Regarding Step 2 of Method Preselect, the identification of attributes depends on both the solution procedures and the performance objectives. For example, for the 0-1 knapsack problem, one relevant attribute for some enumeration procedures is the relationship between the values and sizes of the items. For procedures that use a linear programming bound, as the value to size ratio becomes more uniform,

Figure 1. Overview of Method Preselect.



this bound becomes weaker. This implies that the procedure becomes less efficient. However, this attribute is less relevant for most dynamic programming procedures, which enumerate states and do not use information about item ratios to reduce the search space. Nonetheless, this attribute can be used in selecting between the two types of procedures mentioned because it predicts the performance of one of the procedures.

If conditions are known under which a particular procedure works well or poorly, they can provide a guide to the selection of attributes. For example, suppose that an optimization procedure uses a dominance rule for problem reduction. Then, an appropriate attribute may involve the extent of problem reduction for a given problem instance. For example, for the 0-1 knapsack problem, if the size of item i is smaller than the size of j and its value is larger

than that of j , then i is in the knapsack whenever j is. Solution procedures using branch-and-search are faster for problem instances where more pairs of items satisfy these dominance criteria. Also, in many problems, heuristics find optimal solutions for special cases. Finding attributes that closely describe these cases may help to determine when the heuristic procedure is most effective. An example is bin packing, where all items are small. For this case, the first fit heuristic finds near optimal solutions (Coffman et al. 1996). Consequently, an attribute related to the maximum item size may reliably predict when the first fit heuristic performs well for a given data set.

Some attributes of the data may be hard to specify. This may occur when the relationship between problem data and solution procedure performance is complex. Consider, for example, the satisfiability problem. It is reasonable to believe that the number of clauses and the clause sizes affect problem difficulty. However, these relationships are not clear. Thus, an ad hoc approach may be necessary to identify suitable attributes. Such an approach involves examining random data sets to identify performance patterns. An example of this type of approach is presented in Selman et al. (1996). They generate a wide variety of problem instances for the satisfiability problem and solve them using the procedure of Davis and Putnam (1960). They find that, for fixed clause length models, most instances are easy to solve. However, more difficult instances occur when there are about 4.3 times as many clauses as variables.

Regarding Step 3 of Method Preselect, while measures quantify attributes, they also depend on the performance objective and the procedure being considered. For example, we may wish to select an optimization procedure that has minimum average computation time or one that terminates within a given time with highest probability. In the first case, we may choose a continuous scale measure that predicts average computation time. In the second case, we may prefer a binary or categorical measure that predicts “short” versus “long” computation times. Also, the measures may be correlated or independent. For example, for the attribute graph density, the two measures maximum node degree and number of arcs are positively correlated. This information is important when trying to calibrate the measures to the performance objectives. An example of a measure that depends on the procedure being tested is knapsack capacity for the 0-1 knapsack problem, as discussed in §2.

To calibrate the measures in Step 4 of Method Preselect, a variety of data sets need to be constructed. These data sets should have properties relating to the purpose of the experiment and the procedures being tested. If the purpose is to compare several procedures, then a wide variety of types of data sets is needed. However, if the purpose is to determine whether a given procedure performs well for a specific application, then a large number of data sets with characteristics that are similar to the application are required. Guidelines for choosing an appropriate data set for use in Step 4 are provided by Hall and Posner (2001).

Also, it is important to run enough tests to account for significant but infrequently occurring events. Estimating the number of tests to run is a well-studied problem in experimental design (Cox 1958). Further, creating data sets where the measure values are systematically varied can increase the precision of the estimate for the performance objective (Cox 1958).

Once the data is generated, there are two classes of methods that can be used to calibrate the measures in Step 4. In the first class, the calibration process in Step 4 is completed before the development of decision rules in Step 5. In the second class, the calibration of measures in Step 4 occurs simultaneously with the development of decision rules in Step 5. An example of the first class is multiple linear regression (Neter et al. 1996), where the independent variables are the measures and the dependent variable is the performance objective. The predictive model of performance for each procedure is determined individually. Attention must be given to issues of multicollinearity where measures are correlated. Regression analysis can be used to calibrate measures in various ways. When the process works well, the measures provide a high degree of predictability of solution procedure performance. The use of *t*-statistics provides valuable information about the significance of various measures in predicting performance. Measures that do not help in the prediction process should be discarded, which simplifies the development of decision rules in Step 5.

An alternative to standard regression techniques is support vector regression (Smola and Schölkopf 2004). The idea of this approach is to identify a function that has deviation of at most ϵ from all the observed data and is as simple as possible. In the context of preselecting a solution procedure, a natural interpretation of ϵ is that it represents an insignificant amount of variation in performance. In the context of Method Preselect, obtaining a simple predictive function is important in the development of simple decision rules in Step 5.

Because the experimental data set used to calibrate measures in Step 4 is likely to be more structured than real-world data sets, accurate interpolation is important in solution procedure preselection. An important method for interpolation is kriging (Oliver and Webster 1990). An advantage that kriging has over other interpolation methods is that it assigns weights to observations based on the structure of the local statistical variation, instead of using an arbitrary weighting function. This structure is identified by a variogram, which relates the semivariance of two consecutive data points to their distance.

Another approach that also falls into the first class is categorizing the measures into various levels and then evaluating the performance of solution procedures for the various levels of each measure or combination of measures. Analysis of variance (Sahai and Ageel 2000) can sometimes be used to evaluate the significance of various measures. This approach is especially valuable if the number

of measures is small, or if the performance objectives are not average values. For example, suppose that the objective is to select the procedure that works best most frequently. Then, calibrating the procedure with time is not especially valuable. It is more important to find ranges of measure values where a particular procedure has better performance for many instances. Another case when this approach is useful is when some of the performance values are systematically truncated. An example is where computation times are restricted by an upper bound. In such situations, the above procedures are often unreliable. Because the goal is to achieve accurate performance estimates, the choice of method to use within the first class can be different for different procedures.

For the second class of methods, an example is neural networks (Davallo and Naim 1991). Neural networks are capable of extracting patterns and detecting complex trends. These capabilities can be applied directly to identify the best solution procedure for a given problem instance.

Also in the second class of methods is discriminant analysis (Huberty 1994). An important special case of this method is linear discriminant analysis, which is appropriate when the dependent variable has only two possible categories. An example occurs where the computation time of a particular solution procedure is either “short” (in which case we choose this solution procedure) or “long” (in which case we choose another). If there are more than two categories, then multiple discriminant analysis can be used. Logistic regression analysis (Kleinbaum and Klein 2002) can be used similarly. Press and Wilson (1978) argue for the use of discriminant analysis when the assumptions of multivariate normality are met, and for the use of logistic regression analysis otherwise. It is important to note that both discriminant analysis and logistic regression analysis can also be used within the first class of methods. In this case, these methods would describe results for each procedure individually, but would not directly define decision rules.

In Step 5 of Method Preselect, the results that are collected by the first class of methods in Step 4 are used to develop decision rules for selecting solution procedures. The methodology that is used depends on the type of results that are obtained. For example, when reliable estimates for the computation time of each solution procedure are available, it is easy to select the solution procedure with the best predicted computation time.

In other situations, the performance of two or more solution procedures is highly competitive, or the performance of one or more solution procedures has large variance or is highly skewed. Then, the statistical results collected in Step 4 may provide low predictive power. This situation requires an informal approach to developing decision rules in Step 5. For such cases, we propose an experimental process that involves searching for combinations of measure values which define regions where one optimization procedure is expected to perform better than others. The

use of binary search over the measure values enables the boundaries of such regions to be found efficiently. Additional testing at the boundaries is frequently appropriate. It is important to note that the choice of these regions depends on the performance objectives. For example, one procedure may outperform another in 95% of the instances tested. However, the remaining instances may include some where the first procedure performs much worse than the second one. If the performance objective is frequency of good performance, then the first procedure would be selected. However, if the performance objective is mean performance, then the second procedure may be selected. See §6 for an example and further discussion.

The decision rules developed in Step 5 need to be complete in the sense that, for any given problem instance, they offer an unambiguous recommendation about which optimization procedure to use.

After the decision rules are constructed, they should be validated in Step 6 by computational testing on a random set of problems. In general, a wide selection of new test problems should be generated. Because the original test problems are the ones used to calibrate the measures, they should not also be used to determine the effectiveness of the preselection method. Doing so would probably overestimate the effectiveness of the method. One possibility when discrete measure values are used for calibration is to allow the test data to interpolate between those values.

Further, while several decision rules may be valid, it should be determined whether they are all needed. For example, while Rule A may be useful for certain data sets, other rules may make the same predictions either individually or collectively. In this case, Rule A can be eliminated.

In Step 7 of Method Preselect, given an instance of the optimization problem, we first calculate the measure values for the relevant attributes. These measure values are used to select a particular solution procedure. This selection should be more effective overall than either using random selection or using any specific solution procedure.

4. Definitions and Algorithms for the 0-1 Knapsack Problem

In this section, we introduce the 0-1 knapsack problem and describe two generic algorithms for solving it. Section 4.1 presents some notation and formally defines the problem. Section 4.2 describes a dynamic programming procedure, and §4.3 describes a branch-and-search procedure.

4.1. Notation and Problem Formulation

Let $N = \{1, \dots, n\}$ denote a set of items that can be included in a knapsack of size b . Let c_i denote the value and a_i the size of item $i \in N$. Unless otherwise specified, we assume that the items are indexed by nonincreasing c_i/a_i value. The objective of the 0-1 knapsack problem is to select a subset of the items that has maximum total value,

while satisfying the knapsack size constraint. This problem can be formulated as

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^n c_i x_i \\ & \text{subject to} && \sum_{i=1}^n a_i x_i \leq b, \\ & && x_i \in \{0, 1\}, \quad i = 1, \dots, n. \end{aligned} \quad (1)$$

The 0-1 knapsack problem is binary NP-hard (Karp 1972). Kellerer et al. (2004) provide a comprehensive survey of this and closely related problems. A linear relaxation of this problem is obtained by replacing constraints (1) with $0 \leq x_i \leq 1$ for $i = 1, \dots, n$. If there is no integral solution to the linear relaxation, then there exists a solution with a unique fractional item, f , where $0 < x_f < 1$. Let $\mathcal{L}(F, j)$ denote the optimal solution to the linear relaxation that selects all items in set F , and also selects items $j, j+1, \dots$, until the knapsack is full. The last item selected may be fractional. Let $v(F, j)$ denote the value of $\mathcal{L}(F, j)$.

4.2. Dynamic Programming Algorithm

DP

Input: a_j, c_j for $j = 1, \dots, n$, and b .

Indexing: Index the items in nondecreasing order of a_j .

Value Function: $h(i, g)$ = maximum value of a selection of items $1, \dots, i$ with total size g .

Boundary Conditions: $h(0, 0) = 0$; $h(i, g) = -\infty$ for $g > b$, $i = 0, \dots, n$.

Optimal Solution Value: $\max_{0 \leq g \leq b} \{h(n, g)\}$.

Recurrence Relation: $h(i, g) = \max\{h(i-1, g), c_i + h(i-1, g - a_i)\}$, $i = 1, \dots, n$, $g = 1, \dots, \min\{\sum_{j=1}^i a_j, b\}$.

In the first term of the recurrence relation, item i is not selected. In the second term, item i is selected. While the running time of DP is $O(nb)$, in §5.3 we discuss how to predict the running time more precisely.

4.3. Branch-and-Search Algorithm

The branch-and-search algorithm (BAS) considers the items in nonincreasing order of c_i/a_i . A solution to the linear relaxation of the problem provides an upper bound on the optimal value. If the solution has a fractional item, then it is removed from the selected set. The remaining selected items, with the addition of any items chosen in index order that also fit in the knapsack, comprise the first feasible solution. The value of this solution is a lower bound on the optimal value. If the upper and lower bounds are not equal, then branching occurs by removing the highest indexed selected item from the solution. The above procedure for finding upper and lower bounds is repeated for each subproblem. Further branching occurs until all subproblems are explored.

5. Model Development

In §5.1, we identify characteristics that we believe might affect the performance of BAS and DP. Section 5.2 contains a description of a heuristic procedure for predicting the number of nodes that BAS evaluates. Although this heuristic sometimes overestimates the number of nodes evaluated by BAS, it reliably identifies instances where the number of nodes is large. In §5.3, we describe an extensive computational study that enables us to predict, for a given instance, which algorithm is likely to perform better.

5.1. 0-1 Knapsack Attributes and Measures

To determine the efficiency of a particular optimization procedure for a given data set, we need to identify the attributes of a problem instance that affect the performance of the procedures. Some problem attributes for the 0-1 knapsack problem are:

- (1) problem size,
- (2) characteristics of item values,
- (3) characteristics of item sizes,
- (4) relationships between item value and item size,
- (5) knapsack capacity, and
- (6) characteristics of the linear relaxation solution.

We identify measures that are associated with the attributes listed above. The numbers used for the measures correspond to the above list of attributes.

(1) Number of items, n : gives the maximum height of the enumeration tree for BAS, and is also part of the time complexity of DP.

(2) Coefficient of variation of the c_i values, c_{cov} : large variation in the c_i values provides easier solution discrimination in BAS.

(3) Coefficient of variation of the a_i values, a_{cov} : small variation in the a_i values, especially when combined with small c_{cov} , makes it difficult for BAS to discriminate between solutions.

(4) Proportion of item pairs i, j for which $c_i \geq c_j$ and $a_i \leq a_j$, p_{dom} : an increase in the proportion of dominant pairs increases the closeness of the optimal solution to index order. This makes it easier for BAS to find an optimal solution quickly.

(5a) Knapsack size, b : partially determines the number of calculations at a stage of DP.

(5b) $\sum_{j=1}^{\bar{n}} \sum_{i=1}^j a_i$, where $\bar{n}$ denotes the largest value of j such that $\sum_{i=1}^j a_i \leq b$ when the items are indexed such that $a_1 \leq \dots \leq a_n$: partially determines the number of calculations in DP.

(5c) Knapsack size divided by total item size, size : estimates the proportion of the available items that the knapsack can hold, which affects the performance of BAS.

(6a) Relative percentage gap between the upper bound obtained from linear relaxation and the lower bound obtained from the first feasible solution: a small gap implies fewer solutions that need to be evaluated by BAS.

(6b) Size of the linear programming core: estimates the size of the knapsack core (Balas and Zemel 1980), and is relevant to the performance of BAS.

(6c) Balas-Zemel measure: determines problem difficulty measure Δ (Balas and Zemel 1980), and is relevant to the performance of BAS.

5.2. Heuristic Node Number Estimation

We describe a heuristic procedure, BNODES, that predicts the number of nodes evaluated by BAS. This measure is important because, in our computational tests, the correlation coefficient between ($n \times$ the number of nodes evaluated by BAS) and the computation time of BAS is 0.928. As we discuss in §5.3, BNODES identifies those data sets where BAS evaluates a large number of nodes.

BNODES approximates the actual enumeration tree of BAS. New solutions are eliminated unless the value of the linear relaxation (an upper bound) is larger than the value of the best known integer solution (a lower bound). As in BAS, items can be set to one or zero. Initially, the items in the linear relaxation solution, not including the fractional one, are set to one. Then, we estimate the number of possible noneliminated solutions that are associated with setting a subset of these indices to zero. Items are selected in decreasing index order, rather than considering all possible subsets, which makes the running time of BNODES polynomial. A formal description of BNODES follows.

Heuristic BNODES

Step 0. Reindex the items such that $c_1/a_1 \geq \dots \geq c_n/a_n$.

Find the first integer solution:

- (a) Select items in index order if they fit in the knapsack.
- (b) Continue until the knapsack is full or no items remain.

Let z^0 denote the value of this solution, and let f be the first item not selected.

Set $h_f = 1$.

If $\sum_{i=1}^{f-1} a_i = b$, then stop.

Set $F = \{1, \dots, f-1\}$, $r = f-1$, and $q = f-1$.

Step 1. Find $t_0 = \max\{t \mid v(F, t) > z^0, t \geq f\}$.

If t_0 does not exist, then go to Step 3.

Let $t = \max\{j \mid \text{item } j \text{ is 1 in } \mathcal{L}(F, t_0)\}$.

Starting with item $t+2$, find a feasible integer solution (similar to Step 0).

If the value is larger than z^0 , then replace z^0 with the new value.

Remove item r from F .

Step 2. Set

$$h_n = h_n + \sum_{g=0}^{q-r} \left[\binom{f-r-1}{g} \left(1 + \binom{t-f+1}{g+1} + \sum_{k=1}^{g+1} \binom{t-f-g+k}{k} \right) \right].$$

Set $r = r-1$.

If $r = 0$, then stop. Otherwise, go to Step 1.

Step 3. Add item q to F .

Set $q = q - 1$.

If $q \leq r$, then stop. Otherwise, go to Step 1.

Heuristic BNODES operates in two phases, each consisting of several iterations. The initial value of h_n is one because at least one node is always evaluated.

The first phase starts by finding an initial feasible integer solution. First, $F = \{1, \dots, f - 1\}$. If $v(F, f) > z^0$, then item f is set to zero, and $v(F, f + 1) > z^0$ is checked. The iteration completes when items $f, \dots, t_0$ are set to zero, and $v(F, t_0 + 1) \leq z^0$. At the second iteration, item $f - 1$ is set to zero. This implies that $F = \{1, \dots, f - 2\}$. As in the previous iteration, this iteration completes when items $f, \dots, t_0$ are set to zero, and $v(F, t_0 + 1) \leq z^0$. On completion of each iteration, an estimate of the number of possible solutions that are evaluated under the current configuration is added to h_n . The first phase terminates when either items $1, \dots, f$ are set to zero, i.e., $F = \emptyset$, or $v(F, f) > z^0$ is not satisfied.

In the second phase, some of the items preceding f that are set to zero in the first phase are reset to one in decreasing index order. At the first iteration of the second phase, $F = F \cup \{f - 1\}$. If $v(F, f) > z^0$, then the process continues as in the first stage. Otherwise, the largest index item not in F that has index less than f is set to one, and is added to F . Again $v(F, f) > z^0$ is checked, and this process is repeated until $v(F, f) \leq z^0$. For the next iteration, we set to zero the largest index item in F that has never been set to zero previously. The process now repeats similarly to the prior iteration. On completion of each iteration, an estimate of the number of possible solutions that are evaluated under the current configuration is added to h_n . The second phase terminates when either items $1, \dots, f - 1$ are set to one, or no further items can be removed from F .

At the end of the second phase, h_n provides an estimate of the total number of solutions that are evaluated for all the possible configurations.

Heuristic BNODES runs in low order polynomial time because the search space is reduced by making the following simplifying assumptions:

(a) All items have equal size.

(b) If $t_0 = \max\{t \mid v(F, t) > z^0, t \geq f\}$ and $t = \max\{j \mid \text{item } j \text{ is 1 in } \mathcal{L}(F, t_0)\}$ (see Step 1 of BNODES), then the solution found by any selection of $t - t_0$ items from $\{f, \dots, t\}$ and one item from $\{t + 1, \dots, n\}$ has value no greater than z^0 .

(c) The value of z^0 determined at each stage of BNODES is as large as the corresponding value in BAS.

Assumptions (a) and (b) typically result in only small estimation errors. A potentially more significant source of error is that BNODES may not find the integer solution used by BAS for node elimination, thus violating Assumption (c). Since the integer solution used by BNODES may have a lower value, BNODES may identify more nodes than BAS.

However, the number of nodes actually used by BAS can exceed the prediction by BNODES. It does so in only 14.0% of the randomly generated instances. Moreover, for 99.75% of the instances where the actual number of nodes exceeds the prediction, the actual number is no larger than 1,200. When only a small number of nodes are evaluated, BAS is always at least as efficient as DP. Therefore, the lack of an upper bound on the number of nodes evaluated by BAS for a few instances does not compromise the reliability of our method.

We now establish the correctness of the formula in Step 2 of Heuristic BNODES that is used to update h_n . Observe that when an item is set to zero in BAS, a string of subsequent items is set to one. Only the last node of this string in the branching tree is evaluated and compared with the incumbent value of z^0 . Consequently, each leaf node and each node where an item is set to zero are evaluated.

THEOREM 1. Suppose that Assumptions (a), (b), and (c) hold. Also, suppose that items $r, \dots, q$ can be set to zero, replaced by items from $\{f, \dots, t\}$, and the resulting value of the linear relaxation is larger than z^0 . Then, the number of solutions that BAS checks is

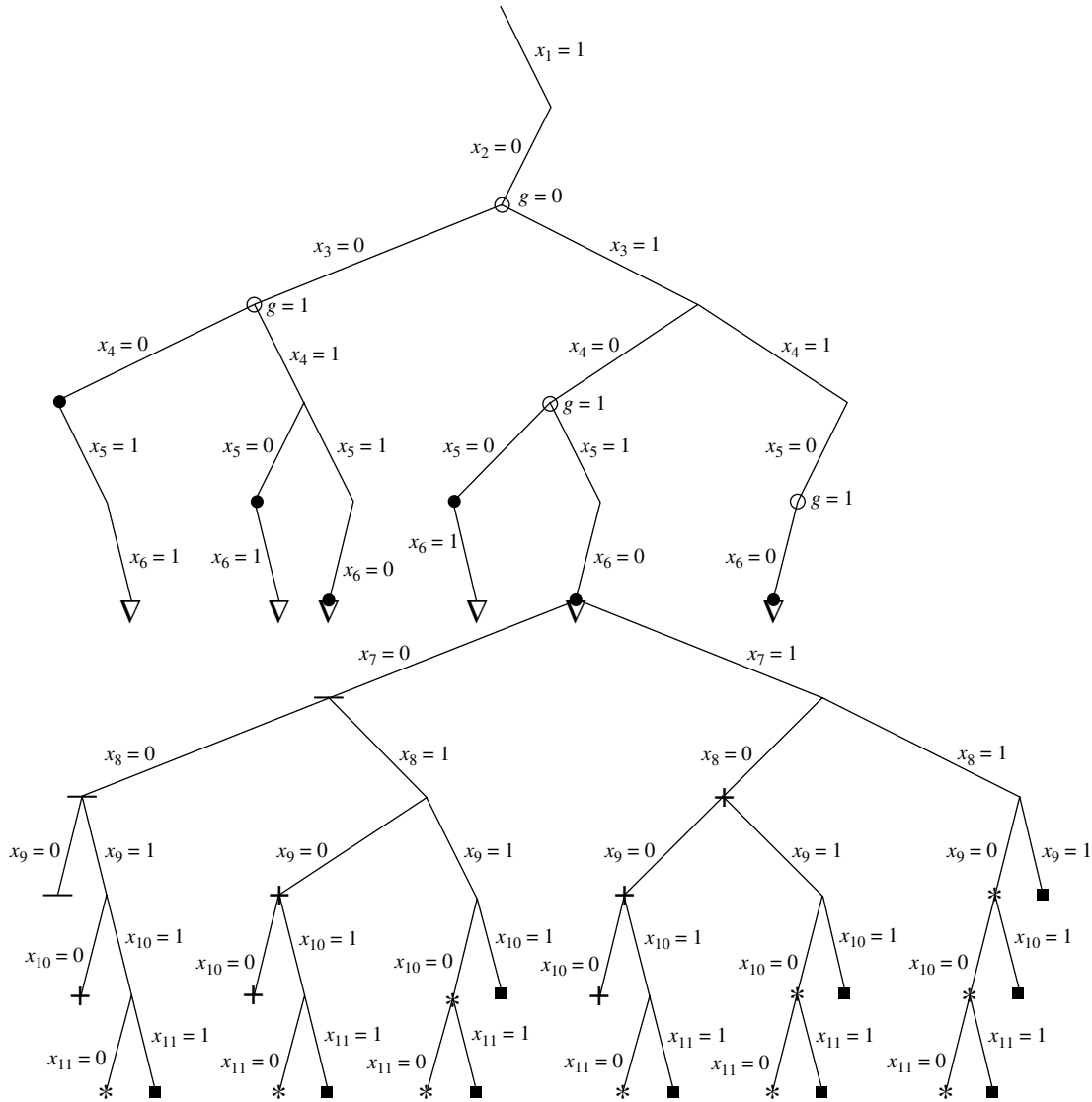
$$\sum_{g=0}^{q-r} \left[\binom{f-r-1}{g} \left(1 + \binom{t-f+1}{g+1} + \sum_{k=1}^{g+1} \binom{t-f-g+k}{k} \right) \right]. \quad (2)$$

PROOF. The calculations are separated into two parts: the selection of items from $\{r, \dots, f - 1\}$ that are set to zero, and the selection of items from $\{f, \dots, t\}$ that are set to one.

In a given iteration, items $\{1, \dots, r - 1\}$ are set to one and item r is set to zero. Each combination of up to $q - r$ items from $\{r + 1, \dots, f - 1\}$ is set to zero. For a given value of g , where $0 \leq g \leq q - r$, this can occur in $\binom{f-r-1}{g}$ ways. Each of these combinations yields one node evaluation (i.e., the last zero) and several node evaluations resulting from setting g items to one from the set $\{f, \dots, t\}$. To avoid double counting, only the last zero from the set of items before f is included. For example, if items 3 and 5 are set to zero, then 3 is evaluated when *only* item 3 is set to zero.

Node evaluation for the items after $f - 1$ that are set to one occurs in two ways. The first is that $g + 1$ items from $\{f, \dots, t\}$ are set to one. This happens in $\binom{t-f+1}{g+1}$ ways. The other possibility is that fewer than $g + 1$ items are set to one. From Assumption (b), this results in the value of the linear relaxation not exceeding z^0 . In this case, $k - 1 \leq g$ items from $\{f, \dots, t - (g - k + 1)\}$ are set to one, followed by one item set to zero. Thus, a total of k items have their value set. This gives $\binom{t-f-g+k}{k}$ node evaluations for any given g and k . $\square$

To illustrate (2), assume that $n \geq 11$ and the fractional item in the linear relaxation is item 7. Consider the subtree where the first item set to zero is item 2, and $v(\{1\}, 8)$

Figure 2. A partial tree evaluated by BNODES, where $g \leq 2$, $r = 2$, $q = 6$, $f = 7$, and $t = 11$.

is larger than the current best integer solution. We calculate the number of nodes found by BNODES when item 2 and two other items from $\{3, \dots, 6\}$ are set to zero. Since two items are set to zero, $g = 2$. The enumeration tree for this case is shown in Figure 2. Further, there are six identical subtrees with branches for $x_7, \dots, x_{11}$. Each subtree attaches at one of the ∇ s. Only the subtree that is connected to the fifth ∇ is shown in detail.

In (2), $\binom{f-r-1}{g} = \binom{7-2-1}{2} = 6$ nodes have a ∇ shown. The first term of $(1 + \binom{t-f+1}{g+1} + \sum_{k=1}^{g+1} \binom{t-f-g+k}{k})$ is shown by the six $\bullet$ s on the nodes of the tree. The second term is shown by $\binom{t-f+1}{g+1} = \binom{11-7+1}{2+1} = 10$ $\blacksquare$ s on the nodes of the tree. For the third term, $\binom{t-f-g+k}{k} = \binom{11-7-2+1}{1} = 3$ nodes corresponding to $k = 1$ have the symbol $-$. Also, $\binom{11-7-2+2}{2} = 6$ nodes corresponding to $k = 2$ have the symbol $+$. Finally, $\binom{11-7-2+3}{3} = 10$ nodes corresponding to $k = 3$ have the symbol $*$.

Even though each terminal node and each node corresponding to some $x_i = 0$ is evaluated at some stage, not all of these nodes are evaluated when $g = 2$. The nodes that are evaluated when $g < 2$ have the symbol $\circ$ and a corresponding value of g .

Next, we determine the computational effort required by Heuristic BNODES. Finding the first integer solution in Step 0 requires $O(n)$ time. Steps 1 and 2 require $O(n)$ and $O(n^2)$ time, respectively. Because Steps 1 and 2 are entered $O(n)$ times, the overall time requirement of BNODES is $O(n^3)$.

The computation time of the measures presented in §5.1 is $O(n^2)$.

REMARK 1. The time requirement to predict the performance of DP and BAS is $O(n^3)$.

5.3. Predicting Relative Performance

To determine the functional relationships between the measures described above and computation times, a large statistical experiment is conducted. The computer program is written and compiled in Microsoft Visual C++, version 6.0, Enterprise edition. Computational testing is performed on a Dell model 8200 personal computer with a 1.8 MHz Pentium IV processor and 512 MB of memory.

In choosing our parameter specifications, we follow the guidelines established by Hall and Posner (2001). In particular, we generate a wide variety of parameter specifications, avoid introducing bias in the creation of dominant pairs by randomly selecting pairs of item costs to swap, and select parameter ranges where the two optimization procedures are competitive. The number of items is 100, 500, or 1,000. The knapsack size is $size \sum_{i=1}^n a_i$, where $size \in \{0.1, 0.3, 0.5\}$. If $size > 0.5$, then it is computationally more efficient to enumerate the items that are not included in the knapsack, which leads to symmetric results. The proportion of dominant item pairs is $kn(n-1)/2$, where $k \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$. The item sizes are generated from an integer uniform distribution, using $a_i \sim UI[LB, UB]$, where $UB \in \{10, 100, 1000\}$ and $LB \in \{0.1UB, 0.4UB, 0.8UB\}$. The item values are generated from an integer uniform distribution, using $c_i \sim UI[LB, 1000]$, where $LB \in \{1, 250, 500\}$. The item values are adjusted, using random pairwise interchanges, as needed to achieve the desired proportion of dominant pairs. The computation time is recorded as 1,000 seconds if a procedure has not terminated by that time.

We construct a full factorial design. The number of possible parameter combinations is $3 \times 3 \times 5 \times 9 \times 3 = 1,215$. For each combination, 10 problem instances are randomly generated, for a total of 12,150 instances. Each instance is solved by BAS and DP.

Because DP does not use bounding, computation times are relatively easy to predict. The worst case time complexity of DP, as discussed in §4.2, is $O(nb)$. However, the regression equation containing only nb and constant terms results in a systematic bias in the error term. Recall that $\bar{n}$ is the largest value of j such that $\sum_{i=1}^j a_i \leq b$. When considering item $i \leq \bar{n}$, it is not necessary to evaluate the state variable g for $g = 0, \dots, b$, but only for $g = 0, \dots, \sum_{j=1}^i a_j$. Therefore, the total amount of enumeration required for an item $i \in \{1, \dots, \bar{n}\}$ is $(\sum_{j=1}^i a_j + 1)$ steps. This suggests that for problems with large variations in the a_i values, it may be advantageous first to sort the items in nondecreasing a_i order.

Alternatively, for items $\bar{n} + 1, \dots, n$, it is necessary to enumerate the state space for $g = 0, \dots, b$. We therefore use $(\sum_{i=1}^{\bar{n}} \sum_{j=1}^i a_j + 1)$ and $(n - \bar{n})(b + 1)$ as independent variables to predict t , the computation time in seconds. The

regression equation

$$t = 0.0556 + 0.00000006 \left(\sum_{i=1}^{\bar{n}} \sum_{j=1}^i a_j + 1 \right) + 0.0000004(n - \bar{n})(b + 1) \quad (3)$$

gives an R^2 value of 97.7%. The t -statistics for the three coefficients are 14.69, 143.85, and 190.05, respectively. These values are all significant at the 0.001 level.

In contrast, computation times for BAS are much less predictable. One measure of the unpredictability of BAS computation times is that R^2 values greater than 0.30 are difficult to obtain. Moreover, BAS computation times are highly skewed, in that the upper tail of the distribution is extremely long. Further, due to time limitations, we only have lower bounds on computation times for some of the data sets. In such situations, the discussion in §3 recommends an experimental process for identifying regions of measures where specific solution procedures should be preselected. Recall from §5.2 that Heuristic BNODES very reliably identifies problem instances where BAS evaluates a large number of nodes. This makes BNODES ideal for defining preselection regions. We implement this in Step A of our preselection algorithm, KPCHOICE, for the 0-1 knapsack problem. Following a description of the algorithm, all the steps are discussed in detail.

KPCHOICE

Rule A. Use BAS if: $nh_n \leq 1.5 \times 10^{11}(0.0556 + 0.00000006(\sum_{i=1}^{\bar{n}} \sum_{j=1}^i a_j + 1) + 0.0000004(n - \bar{n})(b + 1))$.

Rule B. Use BAS if:

- (i) $p_{\text{dom}} \leq 0.7$, and
- (ii) $a_{\text{cov}} \geq 0.25$, and
- (iii) $c_{\text{cov}} \geq 0.15$.

Rule C. Use BAS if:

- (i) $0.35 \leq p_{\text{dom}} \leq 0.65$, and
- (ii) $a_{\text{cov}} \geq 0.10$, and
- (iii) $c_{\text{cov}} \geq 0.15$, and
- (iv) $size \geq 0.20$.

Rule D. Otherwise, use DP.

Our objective is to *minimize the mean computation time* of the preselected algorithm. Alternative objectives, for example, maximizing the number of times the faster algorithm is chosen, would lead to different rules. Rule A compares an estimate of the running time of BAS, based on $O(n)$ computation time at each of h_n nodes, with the estimate of the running time of DP from the regression equation (3). The constant 1.5×10^{11} is set by binary search to minimize the mean computation time. Rule A recommends the use of BAS if the former estimate is smaller than the latter.

There are also several combinations of parameter ranges where only good performance of BAS is observed. To find initial estimates of the boundaries of these ranges, we sequence the results according to the values of the measures

being considered. Then, we compute the average computation time for both algorithms over blocks of 50 consecutive instances. Boundaries are set when consecutive blocks recommend different procedures.

Because accurately fixing the boundaries of the ranges is critical to the success of our method, we perform more detailed computational studies around the boundaries of these ranges. We repeat our initial experiment using increments in parameter values that are 20% of the increments used in the first test.

These additional tests motivate Rules B and C. For $p_{\text{dom}} \leq 0.7$, Rule B identifies instances where neither the value of a_{cov} nor that of c_{cov} is small. A value of a_{cov} above 0.25 limits the number of feasible combinations of maximally selected items. A value of c_{cov} above 0.15 permits easy discrimination between the item values. This rule cannot be extended to values of p_{dom} above 0.7 because the item sizes in the ratio index order tend to increase. This leads to increased computational work to fill the knapsack. Intuition for this situation is provided by the fact that a similar problem occurs with the shortest-processing-time-first heuristic for the partition problem. Rule C identifies a trade-off between the measures, where a smaller value of a_{cov} than in Rule B is compensated by a larger value of c_{cov} . However, these conditions alone are insufficient to prevent poor BAS performance because of the following phenomenon. When the range of the c_i values exceeds that of the a_i values, and an extreme value of p_{dom} occurs, there exists a part of the ratio ordered list in which many consecutive items have similar c_i/a_i ratios. If this part of the list contains item f , then discriminating among the items around f becomes difficult. Therefore, Rule C includes additional conditions on p_{dom} and size that prevent this phenomenon from occurring. The values of the measures used in defining Rules B and C are set experimentally, using binary search over lists of problem instances that are ordered by measure values.

Not satisfying Rules A, B, or C indicates that BAS has a high risk of poor performance relative to DP. In this case, the use of DP is recommended by Rule D. Because the other measures we consider have low statistical significance in predicting which optimization procedure is more efficient, those measures are not represented in KPCHOICE.

6. Testing Model Performance

KPCHOICE is tested on a data set consisting of 5,000 randomly generated instances. The number of items, n , is generated from the integer uniform distribution $UI[100, 1000]$. The value of a_i for $i = 1, \dots, n$ is generated from $UI[a_{\min}, a_{\max}]$, where $a_{\max}$ is first generated from $UI[10, 1000]$, and $a_{\min}$ is then generated from $UI[1, \lfloor 0.8a_{\max} \rfloor]$. The value of c_i for $i = 1, \dots, n$ is generated from $UI[c_{\min}, 1000]$, where $c_{\min}$ is generated from $UI[1, \dots, 750]$. The proportion of dominant pairs, p_{dom} , is generated from the uniform distribution $U[0.1, 0.9]$. Then,

Table 1. Performance of algorithm KPCHOICE.

Algorithm	BAS	DP	KPCHOICE	Best
Mean	35.73	1.35	0.53	0.25
Median	0.10	0.50	0.10	0.10
Maximum	1,000.00	18.00	28.40	12.80

the previously generated c_i values are adjusted using random pairwise interchanges to achieve the specified value of p_{dom} . Finally, the parameter size is generated from $U[0.1, 0.5]$. We set the knapsack size using $b = \lfloor n(a_{\min} + a_{\max})\text{size}/2 \rfloor$.

Computation times for the procedures are summarized in Table 1. The second and third columns show the mean, median, and maximum computation times for the 5,000 instances using BAS and DP, respectively. Similar results using the procedure recommended by KPCHOICE appear in boldface in the fourth column. The last column, “Best,” shows similar results if a perfect preselection of solution procedure is made for all 5,000 instances.

BAS is faster than DP in 3,300 instances. While DP is faster in only 497 instances, it is sometimes much faster than BAS. As can be seen in Table 1, the median computation time of 0.10 seconds for BAS is much less than that of 0.50 seconds for DP. However, the lower bound given by the 1,000 second limit for the mean computation time for BAS is 35.73 seconds, which is much larger than the mean of 1.35 seconds for DP.

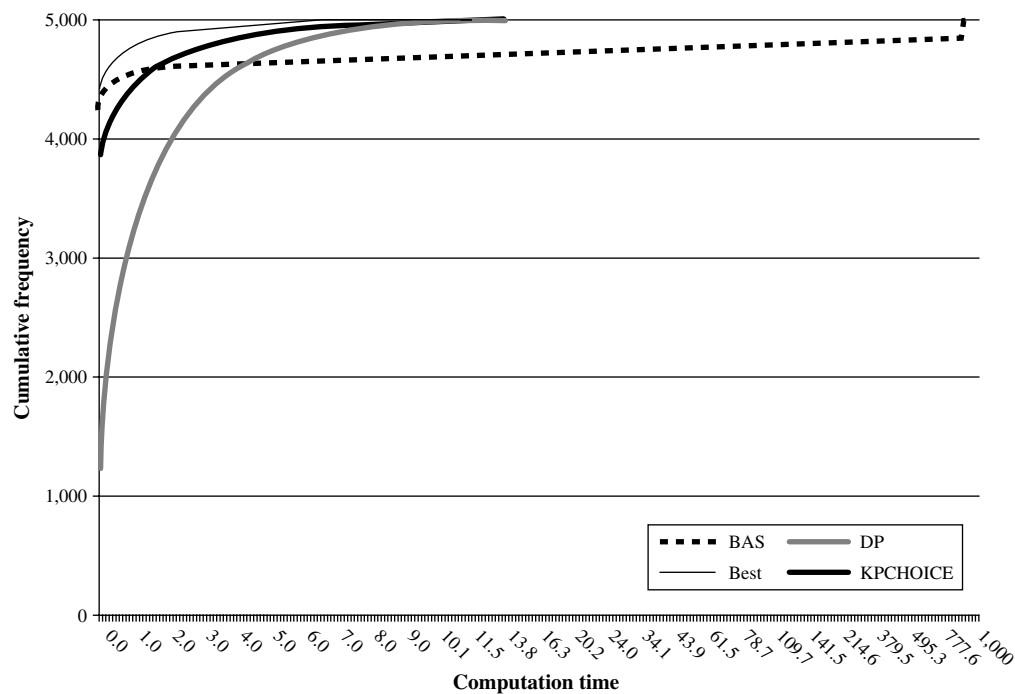
The mean computation time for KPCHOICE is 0.53 seconds. This is a 98.5% improvement over the lower bound on the mean of 35.73 seconds for BAS, and a 60.7% improvement over the mean of 1.35 seconds for DP. However, the best possible mean computation time for a perfect preselection is 0.25 seconds. Therefore, the improvement offered by KPCHOICE over DP represents $100(1.35 - 0.53)/(1.35 - 0.25) = 74.5\%$ of the best possible improvement. Although KPCHOICE is designed to minimize the mean computation time, it also provides the best possible median computation time of 0.1 seconds. Finally, KPCHOICE provides a maximum computation time of 28.40 seconds, which is slightly more than the 18.00 seconds for DP.

In Table 2, each row corresponds to the preselection of a particular procedure by KPCHOICE. For the instances where BAS is preselected, it is faster than DP in 2,613 cases and slower in only 17 cases. However, of the instances where DP is preselected, it is faster than BAS in 480 cases and slower in 728 cases. The latter result follows from our objective of minimizing the mean computation

Table 2. Frequency of correct preselection by algorithm KPCHOICE.

Outcome	Faster	Slower	Tied	Total
Use of BAS	2,613	17	1,041	3,671
Use of DP	480	728	121	1,329

Figure 3. Cumulative frequency of algorithm computation times.



time. The frequent choice of DP is deliberately conservative because the variance of its solution times is much smaller than that of BAS. As discussed in §5.3, a different set of rules would improve performance with respect to the frequency of correct preselection or another objective. The cumulative frequency of computation time for BAS, DP, KPCHOICE, and Best appears in Figure 3. Because KPCHOICE sometimes selects algorithm DP over algorithm BAS to avoid long computation times, BAS outperforms KPCHOICE when computation times are small.

Table 3 provides a breakdown of the decisions in KPCHOICE between Rules A, B, and C. The first row shows the number of instances out of 5,000 for which each rule recommends the use of BAS. The second row, “Sole recommend BAS,” shows the number of instances for which BAS is recommended by only one rule. That is, without the use of that rule, DP would be recommended. Each rule contributes more than 1,000 recommendations, and at least 90 sole recommendations. We therefore conclude that all the rules contribute significantly to the overall performance of KPCHOICE.

Table 3. Contributions of different rules in algorithm KPCHOICE.

	Rule A	Rule B	Rule C
Recommend BAS	3,429	1,510	1,057
Sole recommend BAS	1,703	111	90

7. Concluding Remarks

This paper addresses the issue of whether it is possible, by using only efficiently computable measures of the data, to choose between competing solution procedures. A general methodology is developed to build a predictive model. This methodology is tested for the 0-1 knapsack problem, for which we analyze the performance of two optimization procedures, DP and BAS. For DP, we use problem parameters to predict performance. Also, we identify an important second-order term that improves the predictive power of our model. For BAS, we use both problem parameters and results derived from a heuristic, to obtain a high level of predictive ability. Using a large variety of data sets for which the two optimization procedures are competitive, we demonstrate that our method is consistently able to predict which procedure is more efficient. The use of our preselection method results in considerable saving in mean computation time, compared to the use of either optimization procedure. While a perfect preselection in all instances is not possible, more than 74% of the saving of a perfect preselection is achieved by our preselection method.

Several issues remain open for future research. First, it is useful to determine whether our predictive models for the 0-1 knapsack problem can be improved by the inclusion of other easily computed measures. Second, models can be developed to assess the usefulness of various customizations of optimization procedures for this problem. This would refine the selection process between BAS and DP. An example that is discussed in §5.3 is to determine for

which problem instances the time saving in DP from sorting the items in nondecreasing a_i order exceeds the time required by the sort. Third, our methodology needs to be tested in situations where several optimization procedures are being selected from a larger set. The procedures are implemented in a specified order. When selecting several procedures, it may be valuable to choose ones with negatively correlated performance, in the sense that for problem instances where one procedure performs badly, another should perform well. Fourth, and most importantly, our methodology can be applied to many optimization problems for which alternative procedures, or alternative customizations of a procedure, are available.

We believe that our work impacts the study of optimization in several ways. For many optimization problems, it encourages a detailed understanding of how performance varies with problem measures. Also, it promotes more accurate comparisons between procedures. Finally, it provides a new approach to solving problems more efficiently.

Acknowledgments

This work was supported in part by the National Science Foundation under grants DMI-9821033 and DMI-0421823, and by two grants from the Summer Fellowship Program of the Fisher College of Business (The Ohio State University), to the first author. Expert programming assistance was provided by Kurt Hozak and Muralidhar Krishnamoorthy (The Ohio State University).

References

- Adams, J., E. Balas, D. Zawack. 1988. The shifting bottleneck procedure for job shop scheduling. *Management Sci.* **34** 391–401.
- Angel, E., V. Zissimopoulos. 1998. Autocorrelation coefficient for the graph bipartitioning problem. *Theoret. Comput. Sci.* **191** 229–243.
- Angel, E., V. Zissimopoulos. 2000. On the classification of NP-complete problems in terms of their correlation coefficient. *Discrete Appl. Math.* **99** 261–277.
- Balas, E., E. Zemel. 1980. An algorithm for large zero-one knapsack problems. *Oper. Res.* **28** 1130–1154.
- Bar Yehuda, R., S. Even. 1981. A linear time approximation algorithm for the weighted vertex cover problem. *J. Algorithms* **2** 198–203.
- Cheeseman, P., B. Kanefsky, W. M. Taylor. 1991. Where the really hard problems are. J. Mylopoulos, R. Reiter, eds. *Proc. IJCAI-91*. San Mateo, CA, 331–337.
- Chvatal, V. 1979. A greedy heuristic for the set covering problem. *Math. Oper. Res.* **4** 233–235.
- Coffman, E. G., Jr., M. R. Garey, D. S. Johnson. 1996. Approximation algorithms for bin packing. D. S. Hochbaum, ed. *Approximation Algorithms for NP-Hard Problems*, Chapter 2. PWS Publishing, Boston, MA, 46–93.
- Cox, D. R. 1958. *Planning of Experiments*. John Wiley and Sons, New York.
- Davalo, E., P. Naim. 1991. *Neural Networks*. The Macmillan Press, Ltd., London, UK.
- Davis, M., H. Putnam. 1960. A computing procedure for quantification theory. *J. ACM* **7** 201–215.
- Garfinkel, R. S., K. C. Gilbert. 1978. The bottleneck traveling salesman problem: Algorithms and probabilistic analysis. *J. ACM* **25** 435–448.
- Hall, N. G., M. E. Posner. 2001. Generating experimental data for computational testing with machine scheduling applications. *Oper. Res.* **49** 854–865.
- Hall, N. G., H. Kamoun, C. Sriskandarajah. 1997. Scheduling in robotic cells: Classification, two and three machine cells. *Oper. Res.* **45** 421–439.
- Hill, R. R., C. H. Reilly. 2002. The effects of coefficient correlation structure in two-dimensional knapsack problems on solution procedure performance. *Management Sci.* **46** 302–317.
- Huberty, C. J. 1994. *Applied Discriminant Analysis*. Wiley Interscience, New York.
- Johnson, D. S. 1983. The NP-completeness column: An ongoing guide. *J. Algorithms* **4** 87–100.
- Johnson, D. S., L. A. McGeoch. 1997. The traveling salesman problem: A case study in local optimization. E. H. L. Aarts, J. K. Lenstra, eds. *Local Search in Combinatorial Optimization*. John Wiley and Sons, New York, 215–310.
- Johnson, D. S., L. A. McGeoch. 2002a. Experimental analysis of heuristics for the STSP. G. Gutin, A. Punnen, eds. *The Traveling Salesman Problem and Its Variations*. Springer, New York, 369–443.
- Johnson, D. S., L. A. McGeoch. 2002b. Experimental analysis of heuristics for the ATSP. G. Gutin, A. Punnen, eds. *The Traveling Salesman Problem and Its Variations*. Springer, New York, 445–487.
- Karp, R. M. 1972. Reducibility among combinatorial problems. R. E. Miller, J. W. Thatcher, eds. *Complexity of Computer Computations*. Plenum Press, New York, 85–103.
- Karp, R. M. 1977. Probabilistic analysis of partitioning algorithms for the traveling-salesman problem in the plane. *Math. Oper. Res.* **2** 209–224.
- Kellerer, H., U. Pferschy, D. Pisinger. 2004. *Knapsack Problems*. Springer Verlag, Berlin, Germany.
- Kleinbaum, D. G., M. Klein. 2002. *Logistic Regression*, 2nd ed. Springer, Berlin, Germany.
- Martello, S., P. Toth. 1984. A mixture of dynamic programming and branch-and-bound for the subset-sum problem. *Management Sci.* **30** 765–771.
- Martello, S., P. Toth. 1988. A new algorithm for the 0-1 knapsack problem. *Management Sci.* **34** 633–644.
- Martello, S., P. Toth. 1997. Upper bounds and algorithms for hard 0-1 knapsack problems. *Oper. Res.* **45** 768–778.
- Neter, J., M. H. Kutner, C. J. Nachtsheim, W. Wasserman. 1996. *Applied Linear Statistical Models*, 4th ed. Irwin, Homewood, IL.
- Neto, D. M. 1999. Efficient cluster compensation for Lin-Kernighan heuristics. Ph.D. thesis, Department of Computer Science, University of Toronto, Toronto, Ontario, Canada.
- Oliver, M. A., R. Webster. 1990. Kriging: A method of interpolation for geographical information systems. *Internat. J. Geographical Inform. Systems* **4** 313–332.
- Papadimitriou, C. H., K. Steiglitz. 1982. *Combinatorial Optimization: Algorithms and Complexity*. Prentice-Hall, Englewood Cliffs, NJ.
- Pisinger, D. 1997. A minimal algorithm for the 0-1 knapsack problem. *Oper. Res.* **46** 758–767.
- Pisinger, D. 1999. Core problems in knapsack algorithms. *Oper. Res.* **47** 570–575.
- Potts, C. N., L. N. Van Wassenhove. 1988. Algorithms for scheduling a single machine to minimize the weighted number of late jobs. *Management Sci.* **34** 843–858.
- Press, S. J., S. Wilson. 1978. Choosing between logistic regression and discriminant analysis. *J. Amer. Statist. Assoc.* **73** 699–705.
- Sahai, H., M. I. Ageel. 2000. *The Analysis of Variance: Fixed, Random, and Mixed Models*. Birkhäuser, Boston, MA.
- Selman, B., D. Mitchell, H. J. Levesque. 1996. Generating hard satisfiability problems. *Artificial Intelligence* **81** 111–125.
- Smola, A. J., B. Schölkopf. 2004. A tutorial on support vector regression. *Statist. Comput.* **14** 199–222.
- Szwarc, W., A. Grosso, F. Della Croce. 2001. Algorithmic paradoxes of the single-machine total tardiness problem. *J. Scheduling* **4** 93–104.
- Williams, C., T. Hogg. 1994. Exploiting the deep structure of constraint problems. *Artificial Intelligence* **70** 73–117.